

Table of contents

Part 5 — A one-URL dashboard at <code>http://home.home</code>	1
Step 1 — Deploy Homepage and Portainer	2
Step 2 — Configure the tiles	3
Step 3 — Give the dashboard its pretty URL	6
Step 4 — Try it	6
Troubleshooting	7
Recap	7

Part 5 — A one-URL dashboard at `http://home.home`

The payoff of this part: open any browser on your tailnet, type `http://home.home`, and land on a clean dashboard that shows your services at a glance and links into a full Docker management console — the single front door that ties everything together.

Part 4 gave individual services clean names (`http://pihole.home`, `http://abs.home`). This part adds the *landing page* that gathers them in one place, plus a real management GUI — and it slots into the reverse proxy you built in Part 4 using the exact three-step pattern from the end of that chapter.

We’ll install two complementary tools:

Tool	What it does	Why both
Homepage	A fast, configurable dashboard with live “tiles” for each service	The pretty front door — <i>view</i> status, click through to everything
Portainer	A full web GUI for Docker	The <i>manage</i> half — start/stop/restart containers, read logs, update images, all by clicking

Homepage can *show* container status but can’t *control* containers; Portainer *controls* Docker beautifully but isn’t a customizable landing page. Together they’re the standard beginner homelab combo, and Homepage links to Portainer with a single tile.

i Why Homepage over CasaOS/Cockpit/Homarr

CasaOS wants to own the whole box and install its own Docker stack — it fights the hand-rolled containers you’ve built so far. **Cockpit** manages the host OS, not Docker, so it’s off-target here. **Homarr** is a great click-to-edit alternative if you hate YAML. Homepage wins for a written guide because its config is plain text you can copy-paste, back up, and reproduce exactly.

Step 1 — Deploy Homepage and Portainer

Make the project folder (one stack for the whole control panel):

```
mkdir -p ~/dashboard && cd ~/dashboard
```

Create `~/dashboard/compose.yaml`. Note Homepage has **no host port** — Caddy (from Part 4) is the front door on port 80 and will reach Homepage by container name over the shared `homelab` network:

```
services:
  homepage:
    image: ghcr.io/gethomepage/homepage:latest
    container_name: homepage
    volumes:
      - ./config:/app/config
      - /var/run/docker.sock:/var/run/docker.sock:ro
    environment:
      # Hostnames Caddy will forward here. Comma-separated, NO spaces.
      HOMEPAGE_ALLOWED_HOSTS: home.home,homelab,homelab.your-tailnet.ts.net
      # Widget secrets, injected from .env - never hard-coded in the config
      # files.
      HOMEPAGE_VAR_PIHOLE_PASSWORD: ${PIHOLE_PASSWORD:?set PIHOLE_PASSWORD
in .env}
      HOMEPAGE_VAR_ABS_TOKEN: ${ABS_TOKEN:-}
    networks:
      - homelab
    restart: unless-stopped

  portainer:
    image: portainer/portainer-ce:lts
    container_name: portainer
    ports:
      - "9443:9443" # HTTPS management UI (self-signed cert)
    volumes:
      - /var/run/docker.sock:/var/run/docker.sock
      - portainer_data:/data
    networks:
      - homelab
    restart: always

networks:
  homelab:
    external: true # the shared network created in Part 4

volumes:
  portainer_data:
```

Create this project's `.env` and `.gitignore` — the same pattern as every other stack:

```
cat > ~/dashboard/.env <<'EOF'
PIHOLE_PASSWORD=same-password-you-set-for-pihole
ABS_TOKEN=your-audiobookshelf-api-token
EOF
chmod 600 ~/dashboard/.env

cat > ~/dashboard/.gitignore <<'EOF'
.env
EOF
```

Get the Audiobookshelf token from its web UI under **Settings** → **Users** → **your user** → **API token**, then bring the stack up:

```
cd ~/dashboard && docker compose up -d
```

💡 Two different “reachability” mechanisms

Homepage uses the **Docker socket** to read container *status* (running/stopped, CPU, RAM), and the **homelab network** to call service *APIs* for widget data. The socket gives you the green “running” dot; the network gives you “1,204 ads blocked today.”

i Portainer’s 5-minute clock

The first time you open Portainer (<https://homelab:9443> — accept the self-signed cert warning once), it asks you to create an admin user. Do it **within a few minutes** of starting the container, or Portainer locks itself and you must `docker restart portainer` to reopen the window. The `:lts` image runs natively on a 64-bit Pi.

Step 2 — Configure the tiles

! Why your dashboard looks empty at first

Homepage does **not** auto-discover anything — out of the box it creates *blank* config files and shows a nearly empty page. Everything you see is whatever you put in `./config/*.yaml`. The blocks below are a deliberately *full* starting point — service tiles, a bookmarks row, and an info bar — so the page looks alive immediately.

Homepage reads YAML from the `./config` folder. Replace the auto-generated blank files with these.

`config/settings.yaml` — look and section layout:

```

title: My Homelab
theme: dark
color: slate
headerStyle: boxed
layout:
  Media:
    style: row
    columns: 2
  Network:
    style: row
    columns: 2
  Management:
    style: row
    columns: 2
  Links:                                # a bookmarks group (defined in bookmarks.yaml
below)
    style: row
    columns: 4

```

config/docker.yaml — lets tiles show live container status via the socket:

```

my-docker:
  socket: /var/run/docker.sock

```

config/services.yaml — the tiles, now pointing at the pretty URLs from Part 4:

```

- Media:
  - Audiobookshelf:
    icon: audiobookshelf.png
    href: http://abs.home                # the pretty URL from Part 4
    description: Audiobooks & language courses
    server: my-docker
    container: audiobookshelf           # -> live status dot +
CPU/RAM
    widget:
      type: audiobookshelf
      url: http://audiobookshelf:80      # container name + internal
port (for widget data)
      key: '{{HOMEPAGE_VAR_ABS_TOKEN}}'  # injected from .env - no
real token in this file

- Network:
  - Pi-hole:
    icon: pi-hole.png
    href: http://pihole.home            # the pretty URL from Part 4
    description: DNS ad-blocking

```

```

        server: my-docker
        container: pihole
        widget:
            type: pihole
            url: http://pihole:80 # container name +
internal port
        version: 6 # REQUIRED for Pi-hole v6
(defaults to 5!)
        key: '{{HOMEPAGE_VAR_PIHOLE_PASSWORD}}' # injected from .env - no
real password in this file

- Management:
  - Portainer:
      icon: portainer.png
      href: https://homelab:9443
      description: Manage Docker containers

```

`config/widgets.yaml` — the info bar across the top:

```

- resources:
    cpu: true
    memory: true
    disk: /
- search:
    provider: duckduckgo
    target: _blank
- datetime:
    format:
        timeStyle: short
- greeting:
    text_size: xl
    text: Welcome to the homelab

```

`config/bookmarks.yaml` — a row of quick links (pure links, no Docker logic):

```

- Links:
  - Tailscale admin:
      - abbr: TS
      href: https://login.tailscale.com/admin/machines
  - Router:
      - abbr: RT
      href: http://192.168.1.1
  - Pi-hole docs:
      - abbr: PH
      href: https://docs.pi-hole.net
  - Audiobookshelf docs:

```

```
- abbr: AB
  href: https://www.audiobookshelf.org/docs
```

Apply the config (a plain restart re-reads these files):

```
cd ~/dashboard && docker compose restart homepage
```

The Pi-hole v6 widget gotcha

If the Pi-hole tile shows “API error,” the most common cause is a missing `version: 6` line — the widget defaults to the old v5 API, which no longer exists. The `key` for v6 is your admin password, not a legacy API token.

Step 3 — Give the dashboard its pretty URL

This is the Part 4 pattern, applied to the dashboard. Add a Caddy route and a Pi-hole record so `http://home.home` (and the old `http://homelab`) open the dashboard.

1. Add a block to `~/proxy/Caddyfile`:

```
http://home.home, http://homelab {
    reverse_proxy homepage:3000
}
```

2. Reload Caddy:

```
cd ~/proxy && docker compose restart caddy
```

3. Add a Pi-hole Local DNS record (Settings → Local DNS Records): `home.home` → your Pi’s Tailscale IP (`100.x.y.z`), exactly like the records you added in Part 4.

`Homepage_ALLOWED_HOSTS` is the #1 “blank page” trap

Homepage refuses to render for any hostname not in its allowed list. Caddy forwards `home.home` and `homelab` to it, so both must appear in `Homepage_ALLOWED_HOSTS` (they do, in Step 1). If you add another name later, add it there too and `docker compose up -d --force-recreate homepage` — this variable only takes effect on a recreate, not a plain restart.

Step 4 — Try it

From your laptop or phone, anywhere on the tailnet:

```
http://home.home      # (or http://homelab - both open the dashboard)
```

You should see your dashboard with live tiles for Audiobookshelf and Pi-hole and a link into Portainer. From the Portainer tile you can start, stop, inspect, and update every container by clicking — including pulling new images.

Troubleshooting

`http://home.home` shows a blank page. `Homepage_ALLOWED_HOSTS` doesn't include the host you typed. Add it and `docker compose up -d --force-recreate homepage`.

`http://home.home` times out / “can't connect.” Either the name isn't resolving (check the Pi-hole record from Step 3; try `ping home.home`) or Caddy isn't routing it (confirm the block is in the Caddyfile and you restarted Caddy). `docker logs caddy` shows routing errors.

Widgets show “API error” or stay blank. The widget can't reach the service. Confirm the container is on the `homelab` network (`docker network inspect homelab`) and that you used the container name in the widget url. For Pi-hole, confirm `version: 6` and the correct password.

Portainer won't let me create an admin user. The security timeout elapsed — `docker restart portainer` and reopen `https://homelab:9443` promptly.

Recap

- **Homepage** (no host port) + **Portainer** (on 9443), both on the `homelab` network, deployed as one `~/dashboard` stack.
- **Tiles** point at the pretty URLs from Part 4 and pull live status via the Docker socket and service APIs.
- **Added the dashboard to the proxy** — one Caddy block + one Pi-hole record — so `http://home.home` (and `http://homelab`) open it from anywhere.

Your homelab now has a single, memorable front door. In [Part 6](#) we turn to privacy on the *outbound* side: what a VPN does, how Aura and Mullvad compare, and what a Tailscale *exit node* is.